

Verify the reporting view with real predictions and no odds

Context

`vw_fact_player_prop_odds`, currently defined in `db/migrations/008_checkpoint4_resolution.sql`, inner-joins predictions to players and matches and LEFT JOINS odds. The LEFT JOIN is deliberate: predictions must be visible before any odds exist.

Right now there are zero ATP odds rows, so this is the exact condition the view will run under for a while. It should be proven correct rather than assumed correct. The dashboard reads only this view, so a fault here is invisible everywhere else and shows up as a wrong number on screen.

This is a verification card. Write tests; do not redesign the view.

Tasks

1. Write a test that inserts a real-shaped prediction with no matching odds row, then asserts the row appears in the view with `line`, `over_price`, `under_price`, `edge`, `side`, and `normalized_edge` all NULL. Run it and paste the output.
2. Write a test for row multiplication. Insert one prediction and three odds rows for the same match, player, and prop type from three different books. Assert the view returns exactly three rows, not one and not nine, and that all three carry the same prediction value. Comment in the test why three is correct — one prediction repeated per book so line shopping stays visible.
3. Write a test that a prediction whose match row is missing does not appear. The foreign key on `(match_id, player_id)` should make this impossible to insert at all; if the insert succeeds, the constraint is not doing its job and you should report that as a finding.
4. Test the interval invariant. Attempt to insert a prediction with `lowerci` above prediction, and assert the database rejects it via the `prediction_interval_ordered` CHECK added in migration 013. Attempt a negative `lowerci` and assert the same.
5. Test the edge and side arithmetic with a known case. Given a prediction and a line you choose, assert `edge` equals prediction minus line and that `side` is 'Over' when the prediction is at or above the line. Include the boundary case where prediction exactly equals line, and state in the test which side that is supposed to produce.
6. Run the full test suite with `DATABASE_URL` unset and report the pass, fail, and skip counts. Then report them again with a database available. Both numbers matter — tests that silently skip without a database are a known hazard in this repo.

Do not

Do not change the view's join types. Do not drop or recreate the view. Do not modify the CHECK constraint. If a test reveals the view is wrong, report it as a finding rather than quietly fixing the view in this card.

Report

Report which items you completed and explicitly list any you did not do and why. Do not silently skip items. For each test you wrote, paste the actual assertion and the actual run output — not a summary of it.

What to look for

Item 2 is the subtle one. A LEFT JOIN across three books fans one prediction into three rows, which is intended, but it means any naive COUNT(*) or AVG(prediction) over the view triple-counts. If the dashboard shows aggregate numbers anywhere, that fan-out is a live source of wrong figures, and this test documents the behavior that makes it wrong.

Item 6 exists because a test that skips is not a test that passes. Report the skip count explicitly and name which tests skipped.